SciCalc Next CI/CD Report

Pradyun Devarakonda
IMT2022525
Devarakonda.Pradyun@iiitb.ac.in

October 10, 2025

Quick Links:
GitHub: https://github.com/pradyunuydarp/CI-CD-Learning.git
Docker Hub:
https://hub.docker.com/repository/docker/pradyunuydarp/sci-calc-next/general

1 DevOps Overview

What and Why

DevOps is the practice of combining software development and operations so that product increments can reach users quickly and safely. For this project, DevOps bridges the scientific calculator codebase and the deployment host: every change moves from Git commits through automated tests, container builds, image pushes, and finally to an Ansible-managed runtime. Automating these feedback loops reduces manual effort, ensures reproducibility, and keeps the calculator available.

Toolchain Summary

- **GitHub** — central source control for the repository (CI-CD-Learning) and pull-request history.
- **Node.js 20 & Next.js 15** — application runtime and framework powering the web UI.
- **TypeScript & ESLint** — static typing and linting to catch issues before runtime.
- **Vitest** — unit testing harness for the calculator logic.
- **Jenkins** — orchestrates the CI pipeline across install, test, build, container, and deploy stages.
- **Docker** — packages the Next.js application into a portable image.

- **Docker Hub** — hosts the pushed image under pradyunuydarp/sci-calc-next.
- **Ansible** — pulls the published image and manages the running container on the deployment target.
- **Ansible Vault** — stores Docker Hub credentials for production pulls.
- **LaTeX** — renders this report as a PDF artefact for submission.

2 System Architecture

- Next.js App Router renders the UI under `src/app`. The main calculator form is implemented in `src/components/Calculator.tsx` and rendered by `src/app/page.tsx`.
- Computations live in `src/lib/math.ts` with exported helpers `sqrtX`, `factX`, `lnX`, and `power`. Input validation guards edge cases (negative factorials, non-positive logarithm operands, etc.).
- API requests hit `src/app/api/calc/route.ts`, which accepts JSON payloads, validates the requested operation, and delegates to the math helpers.
- Styling relies on utility classes to keep the layout responsive across laptops and tablets.

3 Implementation Walkthrough

3.1 Source Control Management (GitHub)

The project lives at github.com/pradyunuydarp/CI-CD-Learning. The `sci-calc-next` directory is the Next.js workspace. Typical commands:

```
# clone
$ git clone https://github.com/pradyunuydarp/CI-CD-Learning.git
$ cd CI-CD-Learning/sci-calc-next

# create feature branch, then commit
$ git checkout -b feat/new-capability
$ git add src/app/page.tsx
$ git commit -m "feat(ui): surface factorial operation"
$ git push origin feat/new-capability
```

Jenkins tracks the `main` branch via a GitHub webhook, so every push schedules a pipeline run automatically.

3.2 Testing and Quality Gates

Unit tests use Vitest and live in `src/lib/math.test.ts`. Linting enforces Next.js conventions. Both run locally and in CI:

```
$ npm run lint
$ npm run test
```

The pipeline stops if either command fails, preventing regressions from reaching Docker build or deployment stages.

3.3 Build Automation

The production bundle is created with the standard Next.js build command:

```
$ npm run build
```

In Jenkins, the build stage sets `NODE_ENV=production` inline for the command so that devDependencies remain available during earlier steps.

3.4 Continuous Integration with Jenkins

The Jenkinsfile orchestrates every step. The Install stage now clears leftover dependencies to avoid `ENOTEMPTY` errors before running `npm ci`. Relevant excerpt:

```
stage('Install') {
  steps {
    dir('sci-calc-next') {
      sh 'node -v'
      sh 'rm -rf node_modules'
      sh 'npm ci'
    }
  }
}
```

Dynamic host-port detection prevents deploy clashes, and email notifications keep the team informed. The success/failure handlers share build metadata, Docker tag, and the resolved host port:

```
post {
  success {
    script {
      if (env.NOTIFY_EMAILS?.trim()) {
        emailext(
          to: env.NOTIFY_EMAILS,
          subject: "SciCalc Next build #${env.BUILD_NUMBER} succeeded",
          mimeType: 'text/html',
          body: """
```

```
<p>Build <a href='${env.BUILD_URL}'>#${env.BUILD_NUMBER}</a> completed successfully.</p>
<p>Docker image: ${env.DOCKER_IMAGE ?: 'n/a'}<br/>
Host port: ${env.DEPLOY_HOST_PORT ?: '3000'}</p>
"""
        )
      }
    }
  }
  failure {
    echo 'Pipeline failed. Review the logs for details.'
    script {
      if (env.NOTIFY_EMAILS?.trim()) {
        emailext(
          to: env.NOTIFY_EMAILS,
          subject: "SciCalc Next build #${env.BUILD_NUMBER} failed",
          mimeType: 'text/html',
          body: """
<p>Build <a href='${env.BUILD_URL}'>#${env.BUILD_NUMBER}</a> failed.</p>
<p>Review the <a href='${env.BUILD_URL}console'>console log</a> for details.</p>
""",
          attachLog: true
        )
      }
    }
  }
}
```

During the 10 Oct 2025 run at 16:34 the script detected that port 3000 was occupied, selected 3300 automatically, and completed deployment without manual intervention.

3.5 Containerization

The Dockerfile performs a multi-stage build. Key fragment:

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:20-alpine AS runner
ENV NODE\_ENV=production
WORKDIR /app
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/.next ./.next
```

```
COPY --from=builder /app/public ./public
RUN npm ci --omit=dev
EXPOSE 3000
CMD ["npm", "run", "start"]
```

The Jenkins Docker stage logs into Docker Hub using the `dockerhub` credential and tags the image as `pradyunuydarp/sci-calc-next:latest`.

3.6 Registry Publishing

Image publishing relies on native Docker CLI commands executed from Jenkins:

```
echo "${DOCKERHUB_TOKEN}" | docker login -u "${DOCKERHUB_USER}" --password-stdin
docker build -t "${DOCKERHUB_USER}/sci-calc-next:latest" .
docker push "${DOCKERHUB_USER}/sci-calc-next:latest"
docker logout
```

The resulting image is visible under Docker Hub tags.

3.7 Configuration Management and Deployment

Ansible handles the final delivery. Inventory and group variables pin the container name, port mapping, and interpreter:

```
# ansible/inventory/hosts.ini
[targets]
localhost ansible_connection=local

# ansible/inventory/group_vars/all.yml
app_name: sci-calc-next
docker_image: "{{ lookup('env', 'DOCKER_IMAGE') | default('pradyunuydarp/sci-calc-next:l
container_name: sci-calc-next
container_port: 3000
host_port: "{{ lookup('env', 'HOST_PORT') | default('3000', true) }}"
ansible_python_interpreter: "{{ lookup('env', 'ANSIBLE_PYTHON_INTERPRETER') | default('/
```

The deploy playbook removes stale containers, pulls the requested image, and recreates the service:

```
- name: Stop existing SciCalc Next container
  community.docker.docker_container:
    name: "{{ container_name }}"
    state: stopped
  ignore_errors: true

- name: Remove existing SciCalc Next container
  community.docker.docker_container:
```

```
    name: "{{ container_name }}"
    state: absent
  ignore_errors: true

- name: Ensure SciCalc Next container is running
  community.docker.docker_container:
    name: "{{ container_name }}"
    image: "{{ docker_image }}"
    state: started
    recreate: true
    force_kill: true
    restart_policy: always
    published_ports:
      - "{{ host_port }}:{{ container_port }}"
    env:
      NODE_ENV: production
```

During the successful Jenkins run, the play recap reported `ok=3 changed=1 failed=0`, confirming the container restart on the new port.

4 Application Experience

User Journey

Landing on `/` presents a centered glassmorphism shell with the "SciCalc Next" banner and a two-column layout on desktop. The left pane hosts the interactive form: four quick-action pills switch between square root, factorial, natural logarithm, and power, while an optional free-form expression box accepts inputs such as `5!`, `sqrt(144)`, `ln(10)`, or `pow(2,8)`. Selecting an operation swaps in the relevant structured fields (single numeric input for unary functions, paired base/exponent inputs for power) so the user can either type shorthand math or rely on labelled controls.

Feedback Loop

Submitting the form issues a `POST` to `/api/calc`, which validates the payload and runs the math helpers. Results appear instantly in the right-hand "Result preview" card, formatted with up to six decimal places and wrapped in a scrollable badge for long outputs. A single click copies the numeric answer to the clipboard and surfaces a transient "Copied!" indicator; unsupported clipboard environments fall back to an inline warning instead of failing silently.

Guidance and Validation

Each operation tile advertises its domain (e.g., factorial highlights non-negative integers, logarithm warns about positive inputs only), and the sidebar mirrors the currently selected operation with a description and worked example. Client-side parsing catches malformed

expressions before the request leaves the browser, while the API raises explicit error messages for domain breaches such as negative square roots or overflowing factorials. These messages render in the sidebar with accentuated error styling, ensuring users understand why a computation failed.

Responsiveness and Accessibility

The grid collapses to a single column below 960 px, keeping the keypad, inputs, and feedback stacked for mobile reachability. Buttons expose `aria-pressed` state, results live in an `aria-live` region, and focus-visible styles ensure keyboard navigation remains clear. Color contrast between foreground text and the dark background exceeds WCAG AA targets, making the calculator legible in low-light settings.

5 Runtime Access

A secure tunnel is exposed through ngrok during local demos so remote reviewers can reach the calculator without opening firewall ports.

6 Results and Verification

- **Automated tests**: `npm run test` and `npm run lint` are executed locally and inside Jenkins. Both passed before the most recent push.
- **Pipeline completion**: The 10 Oct 2025 16:34 IST Jenkins run completed every stage, published the Docker image, and redeployed via Ansible using host port 3300.
- **Runtime check**: On the deployment host, `docker ps --filter name=sci-calc-next --format '{{.Ports}}'` shows the chosen port mapping (for example `0.0.0.0:3300->3000/tcp`).
- **Local reproduction**: `npm run lint`, `npm run test`, `npm run build`, and `ansible-playbook playbooks/deploy.yml` reproduce the pipeline sequence offline.

7 References and Access

- GitHub repository: https://github.com/pradyunuydarp/CI-CD-Learning.git
- Docker Hub image: https://hub.docker.com/repository/docker/pradyunuydarp/sci-calc-next/general
- Jenkins job: hosted on the local Jenkins master (`https://<jenkins-host>/job/SciCalc/`).
- Ansible inventory: `ansible/inventory/hosts.ini`
- Jenkins pipeline script: `Jenkinsfile`
- Dockerfile: `Dockerfile`

8 Appendix A: Assignment Brief

Problem Statement

Create a scientific calculator that supports the following operations: square root ($\sqrt{x}$), factorial ($x!$), natural logarithm ($\ln x$), and exponentiation (x^b). The solution may be a command-line or web application in any programming language.

DevOps Requirements

1. Manage the source code with Git on a hosted platform such as GitHub, GitLab, or Bitbucket.
2. Implement automated testing (JUnit, Selenium, Vitest, etc.).
3. Configure a build step using tools like Maven, Gradle, npm, or equivalents.
4. Set up continuous integration with Jenkins.
5. Containerise the application with Docker.
6. Push the resulting image to Docker Hub.
7. Use Ansible for configuration management: pull the image to managed hosts and run the container (local machine or cloud host).
8. Document the entire pipeline and provide screenshots demonstrating each stage and calculator functionality.

Submission Expectations

- Report on the DevOps approach, tools, configuration steps, commands, and screenshots of key milestones (Git commits, Jenkins credentials, Docker Hub repository, Ansible execution, and application UI).
- Provide artefacts such as the Dockerfile, Ansible playbooks, Jenkinsfile, and application source code.
- Include links to the public GitHub repository and Docker Hub image in the report.
- Export the final document as PDF named with the roll number.

9 Appendix B: Screenshot Gallery

The figures below showcase core artefacts captured during the project.

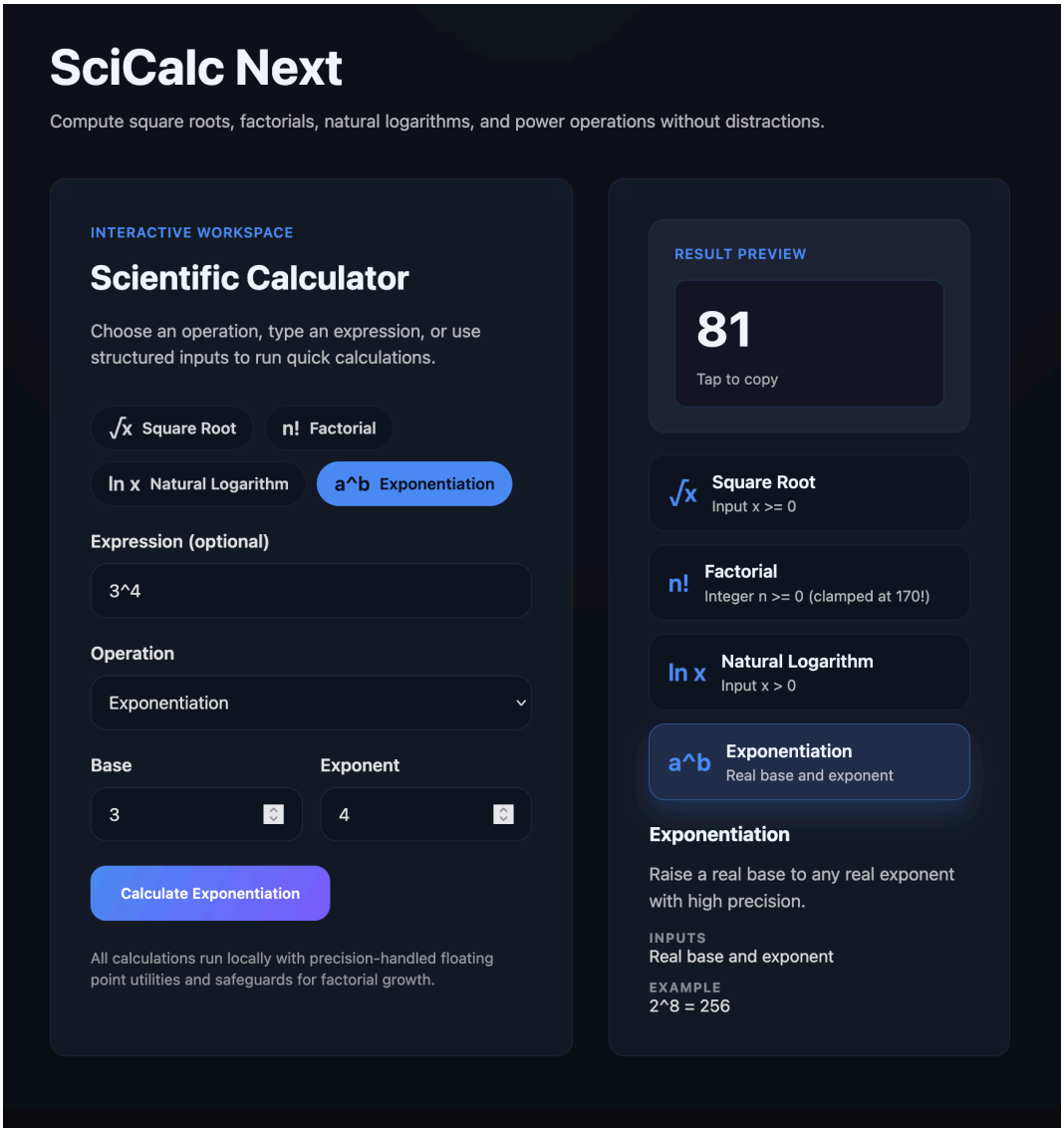

Figure 1: Scientific calculator UI demonstrating square root, factorial, logarithm, and power operations.

Figure 2: Jenkins pipeline stage view highlighting the passing Install, Test, Build, Docker, and Ansible stages.

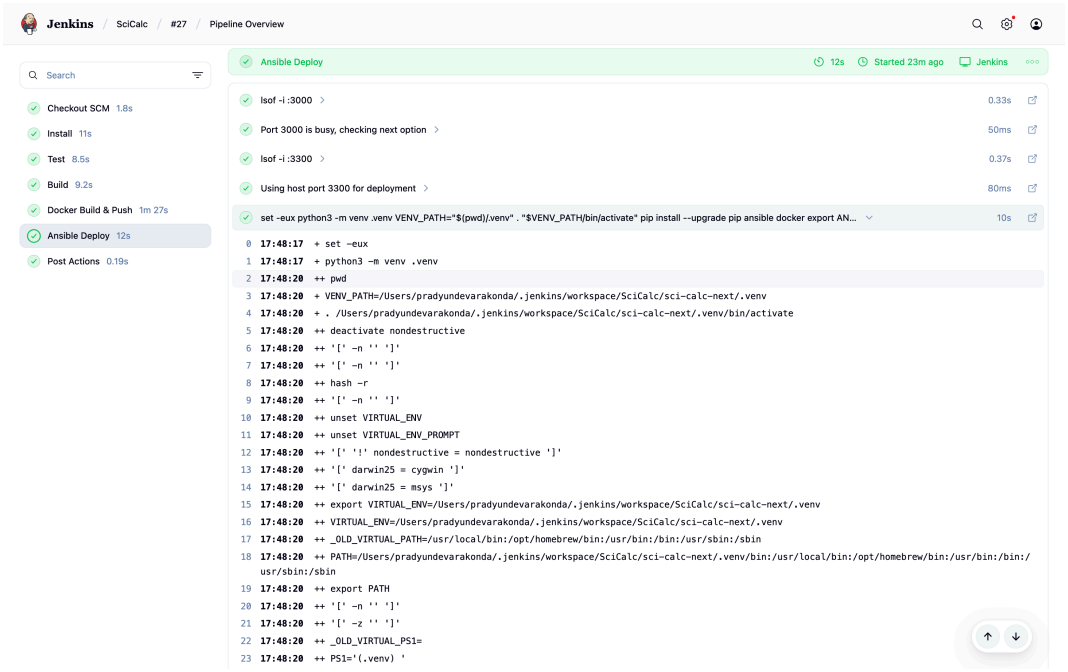

Figure 3: Ansible play recap from Jenkins showing the container restart with host port realignment.

10 Appendix C: Command Reference

```
# Local development
npm install
npm run dev

# Quality gates
npm run lint
npm run test

# Build and containerise
npm run build
docker build -t pradyunuydarp/sci-calc-next:dev .

# Run Ansible locally
cd ansible
ansible-playbook playbooks/deploy.yml

# Inspect deployed container
docker ps --filter name=sci-calc-next --format '{\{\{.Ports\}\}}'
```

11 Learnings and Improvements

- Automated port detection in Jenkins prevented repeated binding errors once multiple containers were active on the same host.
- Exporting the virtual environment interpreter resolved "module not found" issues for `community.docker` modules.
- Future enhancements include Playwright smoke tests, caching the Ansible virtual environment between Jenkins runs, and promoting tagged releases through a staging environment before production.